Secure Software Design and Development
(CY-321)

Web Security in Real World

The Browser Architecture: Overview

The browser is the core client-side component. Its counterpart is the web server.

Browsers are complicated

- Large and complex codebase
- Network-visible interface
- Written in memory-unsafe languages
- Lots of new features added regularly

Without a solid architecture, this is a recipe for disaster!

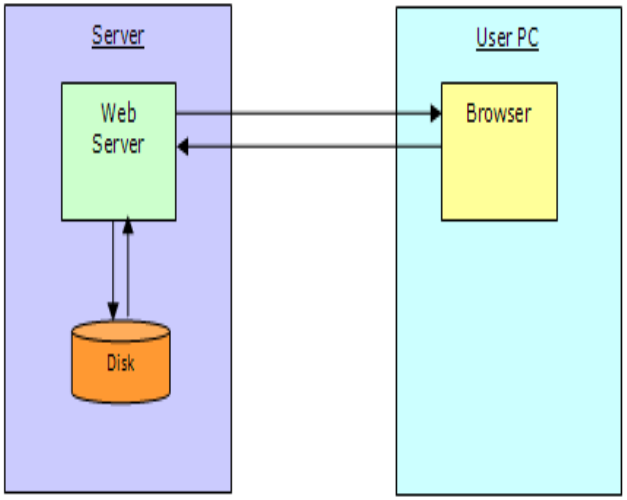
The Browser Architecture: Overview

- Overview of the communication between the web servers and web browsers.

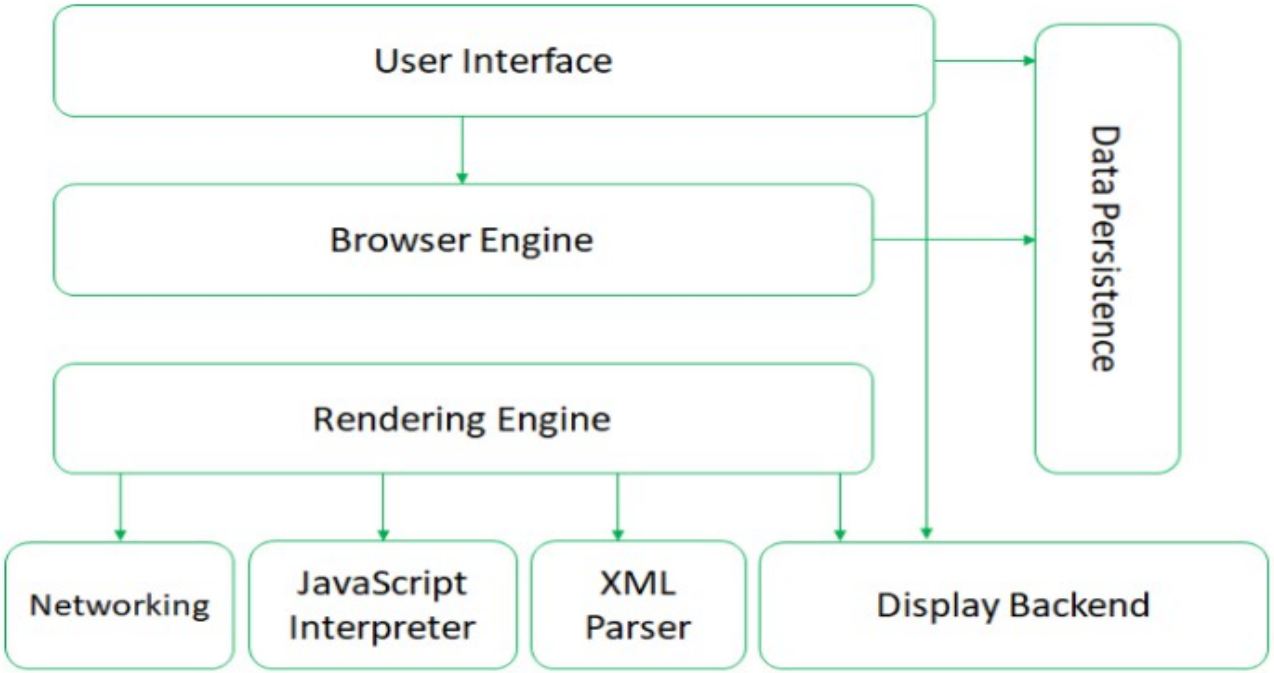
- The web server and a browser typically running in separate machines.

- The browser only has access to the local disk by asking the user for permission

- The web server and the browser communicate over the network.

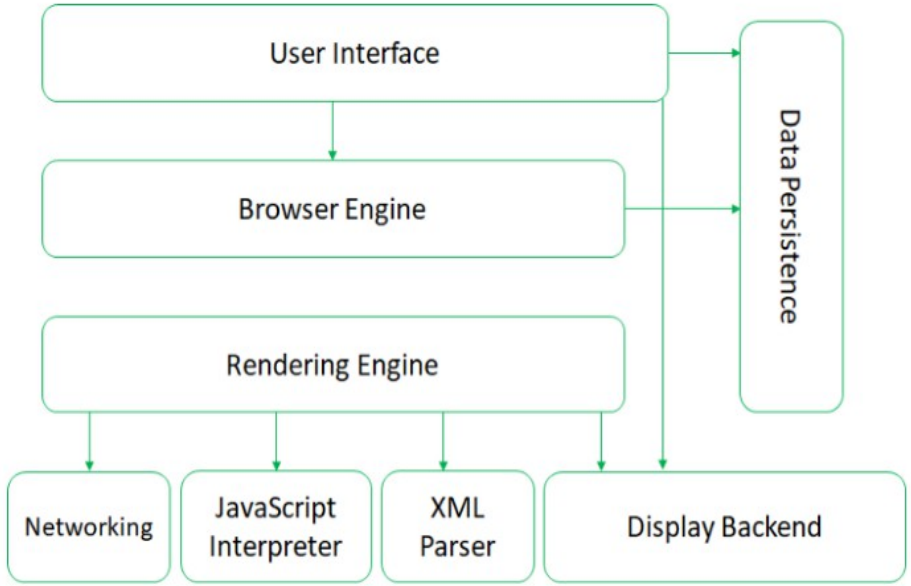


The Browser Architecture: Overview



Architecture of Web Browsers (Reference Architecture)

The Browser Architecture: Overview



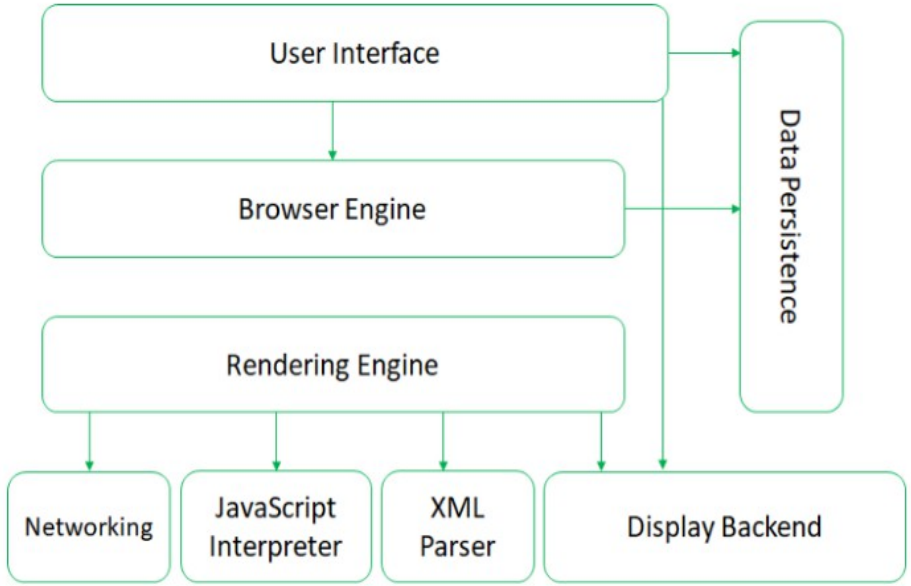
Architecture of Web Browsers (Reference Architecture)

The User Interface subsystem

-may be integrated with the desktop environment to provide browser session management

-provides features such as toolbars, visual page-load progress, smart download handling, preferences, and printing.

The Browser Architecture: Overview

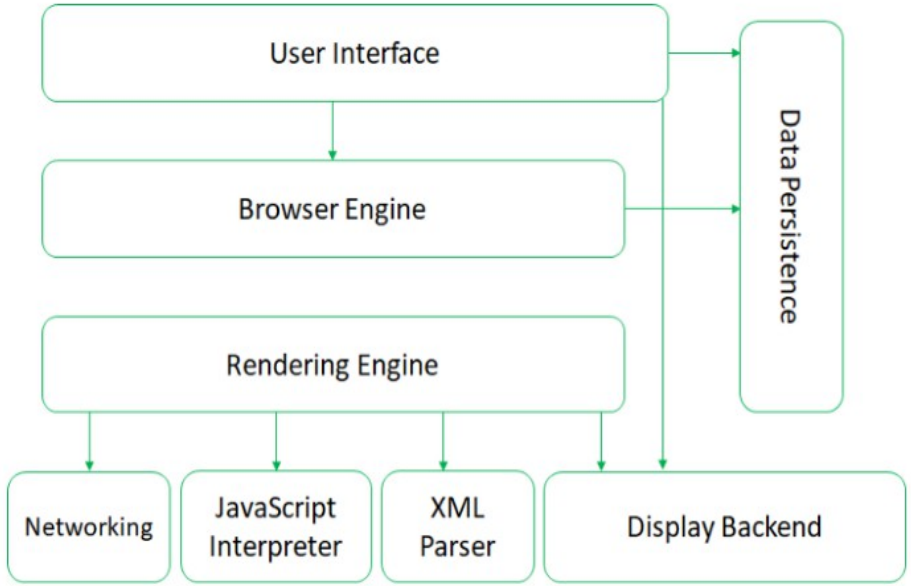


Architecture of Web Browsers (Reference Architecture)

Rendering Engine

-a high-level interface to the Rendering Engine.
-loads a given URL and supports primitive browsing actions such as forward, back, and reload.
-provides hooks for viewing various aspects of the browsing session such as current page load progress and JavaScript alerts.
-allows the querying and manipulation of Rendering Engine settings.

The Browser Architecture: Overview

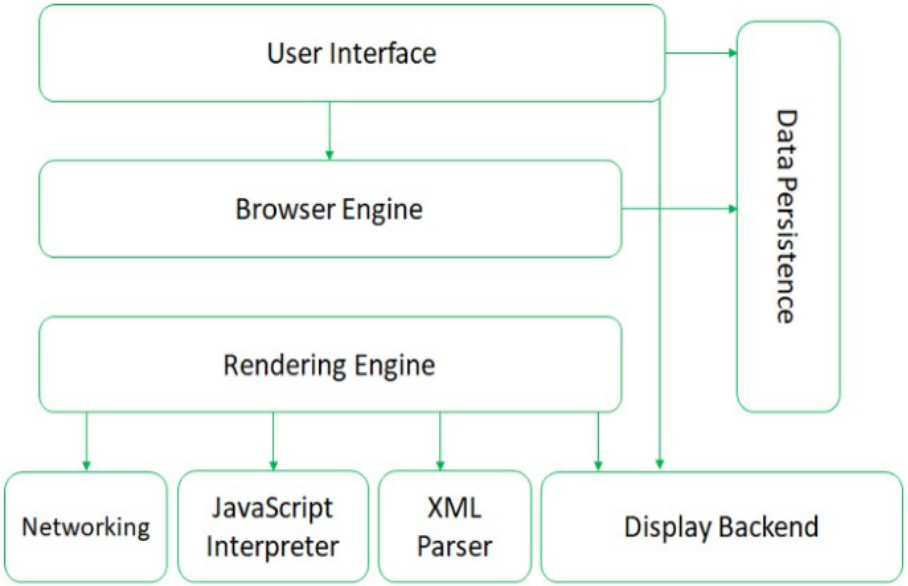


Architecture of Web Browsers (Reference Architecture)

Browser Engine

-visual representation for a given URL.
-displaying HTML and XMLdocuments, optionally styled with CSS, embedded content (images)
-IE(Trident), Firefox(Gecko), Safari, Chrome and Opera(Webkit). -Chrome runs various instances of this engine with various tabs.

The Browser Architecture: Overview

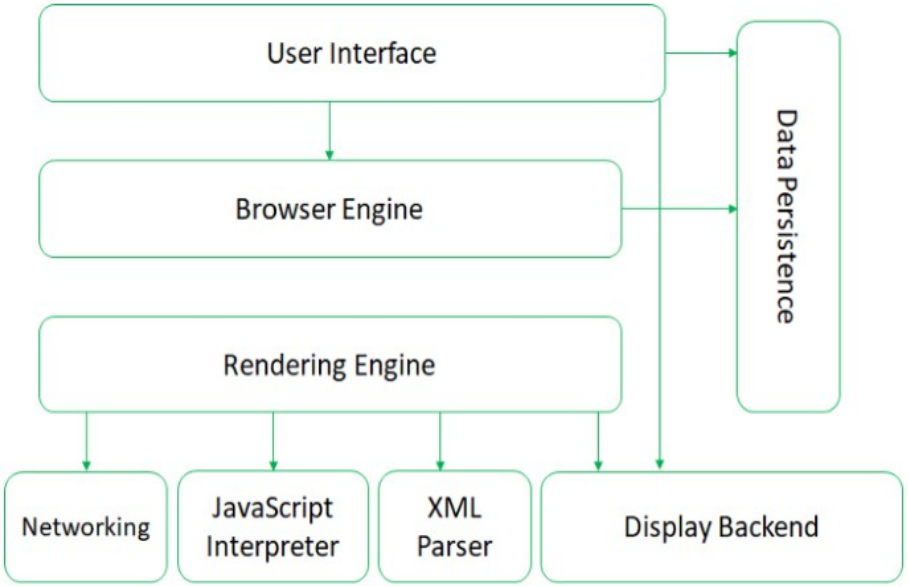


Architecture of Web Browsers (Reference Architecture)

Networking subsystem

-implements file transfer protocols such as HTTP

The Browser Architecture: Overview

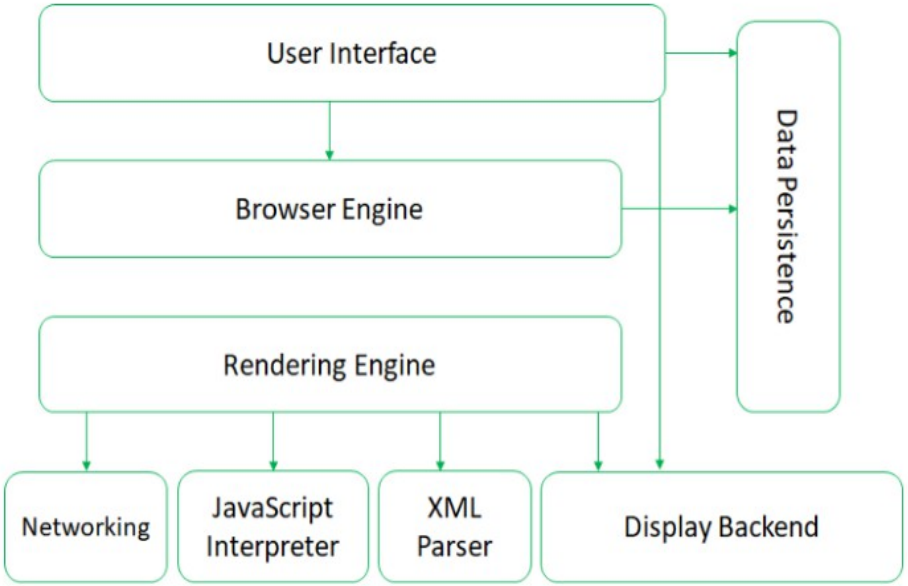


Architecture of Web Browsers (Reference Architecture)

JavaScript Interpreter

-evaluates JavaScript code, which may be embedded in web pages.

The Browser Architecture: Overview

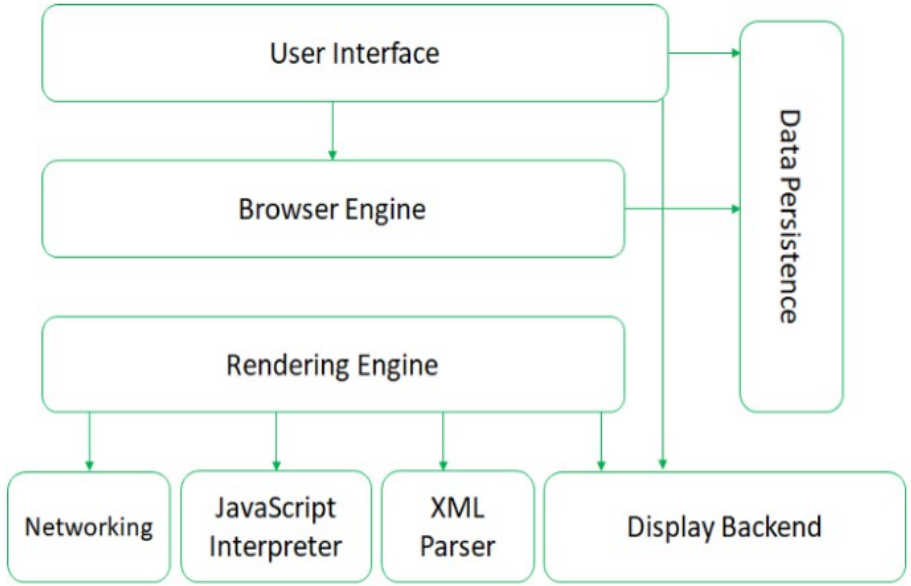


Architecture of Web Browsers (Reference Architecture)

XML Parser subsystem

-parses XML documents into a Document Object Model (DOM) tree[2]. -The most reusable subsystems in the architecture.

The Browser Architecture: Overview

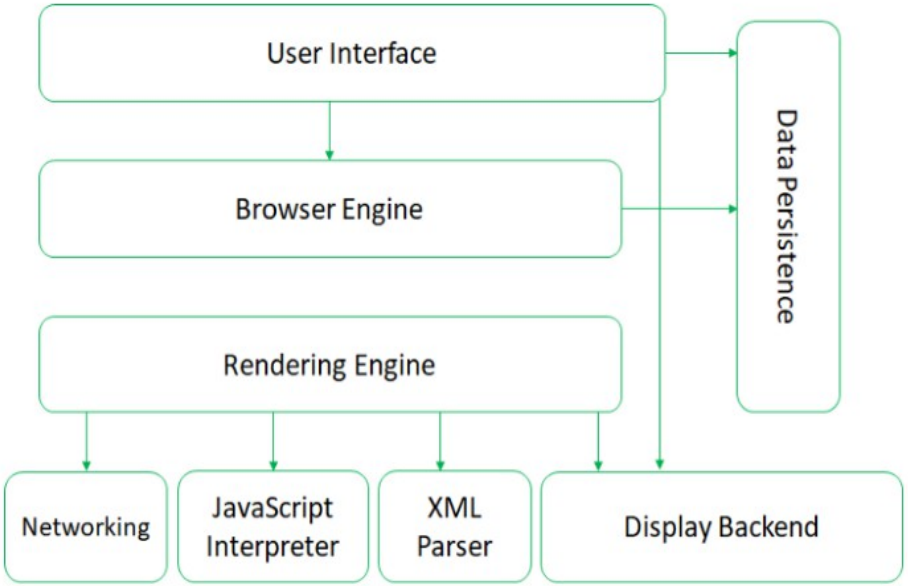


Architecture of Web Browsers (Reference Architecture)

Data Persistence

- stores various data associated with the browsing session on disk.
- high-level data such as bookmarks or toolbar settings,
- or it may be low-level data such as cookies, Preferences, security certificates, or cache.

The Browser Architecture: Overview

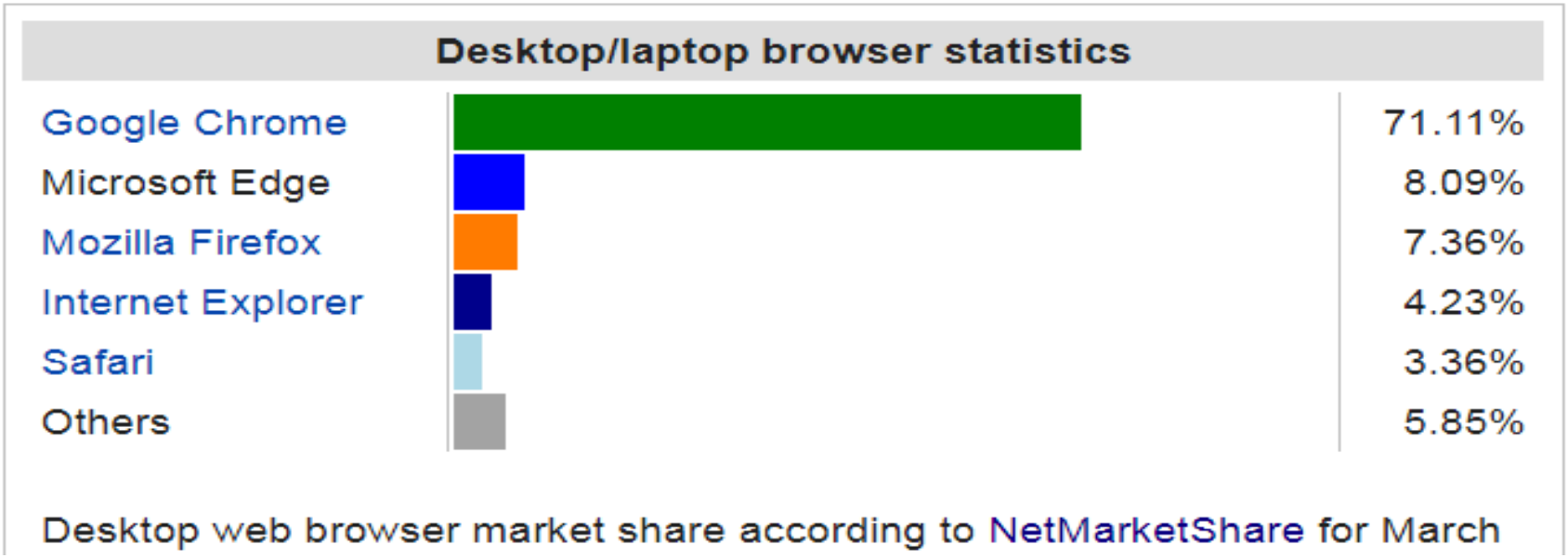


Architecture of Web Browsers (Reference Architecture)

Display Back-end subsystem (UI Backend)

-provides drawing and windowing primitives, a set of user interface widgets, and a set of fonts.

-may be tied closely with the operating system


Desktop/laptop browser statistics
Google Chrome
71.11%
Microsoft Edge
8.09%
Mozilla Firefox
7.36%
Internet Explorer
4.23%
Safari
3.36%
Others
5.85%
Desktop web browser market share according to NetMarketShare for March

Why to worry for Browsers Security

A browser often connects to more than the one address Fetching data can entail accesses to numerous locations

Browser software can be malicious or can be corrupted to acquire malicious functionality.

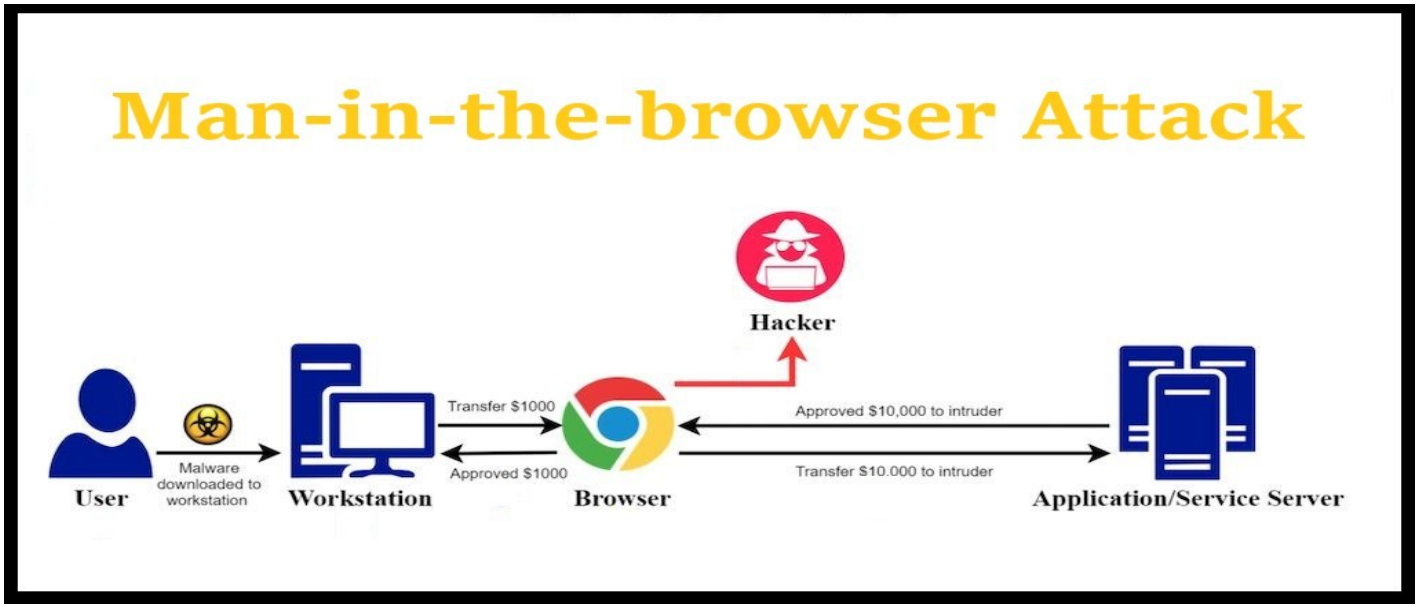
Browsers support add-ins, extra code to add new features to the browser, but these add-in can include corrupting code.

Data display involves a rich command set that controls rendering, positioning, motion, layering, and even invisibility.

The browser can access any data on a user computer. The browser runs with the same privileges as the user.

Data transfers to and from the user are invisible, meaning they occur without the user' s knowledge or explicit permission.

Browser Attacks : Three Attacks Vectors



[4]https://www.kratikal.com/There are many examples for man-in-the-browser malware and attack campaigns targeting online banking and other internet services. Infamous names of malware used include: Zeus, Spyeye, Bugat, Carberp, Silon, Tatanga, and more.

Man-in-the-Browser Attack

Man-in-the-browser is a form of man-in-the-middle attack.

An attacker is able to insert himself into the communications channel between two trusting parties by compromising a Web browser.

Purpose is for eavesdropping, data theft and/or session tampering.
attackers to carry out various forms of financial fraud, typically by manipulating Internet Banking Services.

In order to compromise the browser, adversaries can take advantage of security vulnerabilities and/or manipulate inherent browser functionality to change content, modify behavior, and intercept information.

Various forms of malware,e.g. Trojan horse, can be used to carry out the attack.

A page-in-the-middle attack is another type of browser attack in which a user is redirected to another page.

when the user clicks login page of any site, the attack might redirect the user to the attacker's page, where the attacker can also capture the user's credentials.

Program Download Substitution

Coupled with a page-in-the-middle attack is a download substitution.

The attacker presents a page with a desirable and seemingly innocuous program for the user to download, for example, a browser toolbar or a photo organizer utility.

Attack also defeats users' access controls that would normally block software downloads and installations, because the user intentionally accepts this software.

Keystroke Logger

A keystroke logger (or key logger) is either hardware or software that records all keystrokes entered. The logger either retains these keystrokes for future use by the attacker or sends them to the attacker across a network connection

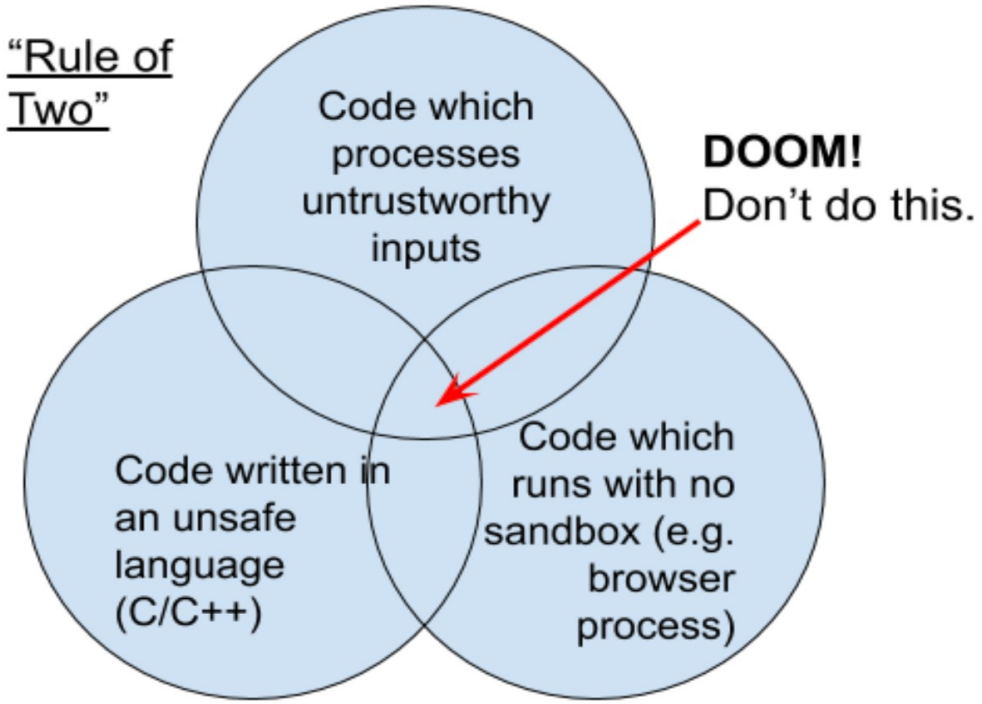

```
let faultyArrayBuffer = new ArrayBuffer(32)
faultyArrayBufier._defineGetter_('byteLength', () => 0xFFFFFFFC)
let faultyBuffer = new Uint32Array(faultyArrayBufier)

console.log(faultyBufier[1000])

faultyBuffer[5000] = 0x42
```

There is lots of additional code required to get RCE, but you get the idea!
See: https://bugs.chromium.org/p/chromium/issues/detail?id=351787

Top goal: Reduce bugs caused by memory unsafety

Recent Microsoft study: "~70% of the vulnerabilities addressed through a security update each year continue to be memory safety issues"

As of March 2019, only about 5 of 130 public Critical-severity bugs are not obviously due to memory corruption


"Rule of Two"
Code which processes untrustworthy inputs
DOOM!
Don't do this.
Code written in an unsafe language (C/C++)
Code which runs with no sandbox (e.g. browser process)

Prototype-based inheritance

- Unlike class-based languages such as Java or C++, JavaScript uses prototype-based inheritance

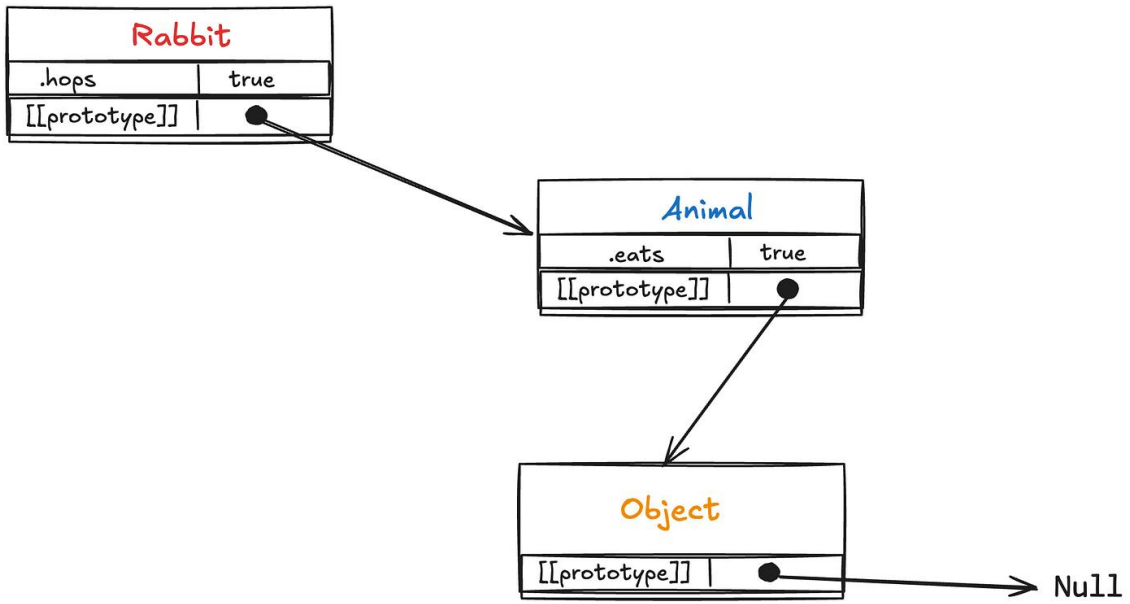
- Every object in JavaScript has an internal link to another object called its prototype

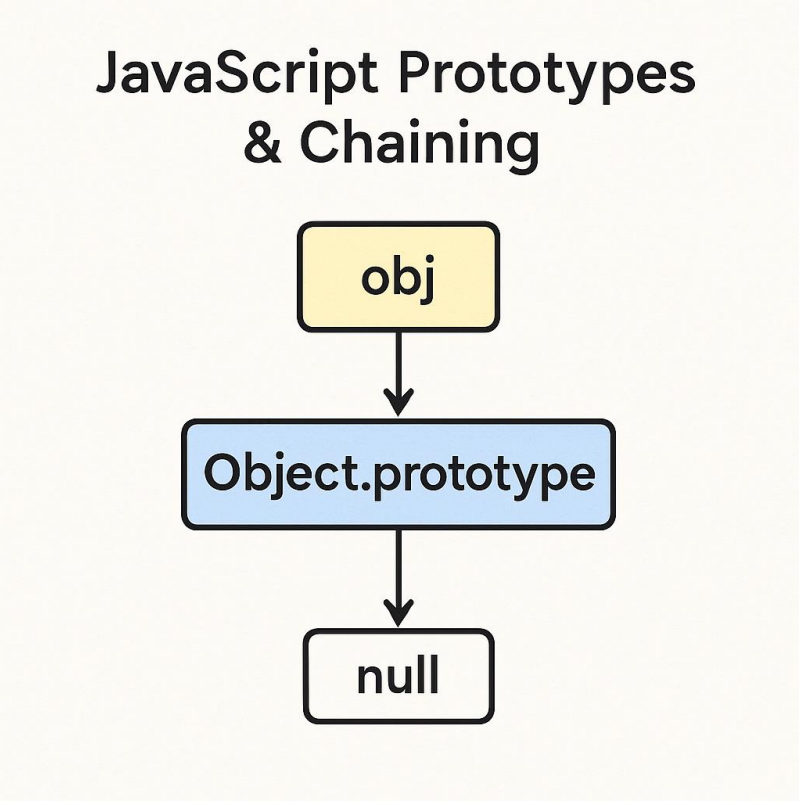
- Prototype-based inheritance means JavaScript objects inherit properties from other objects through a prototype chain.

- Every object in JavaScript can "borrow" features from another object

- If JavaScript cannot find a property in an object, it looks in its prototype. If not found there, it keeps going up the chain.

Prototype-based inheritance





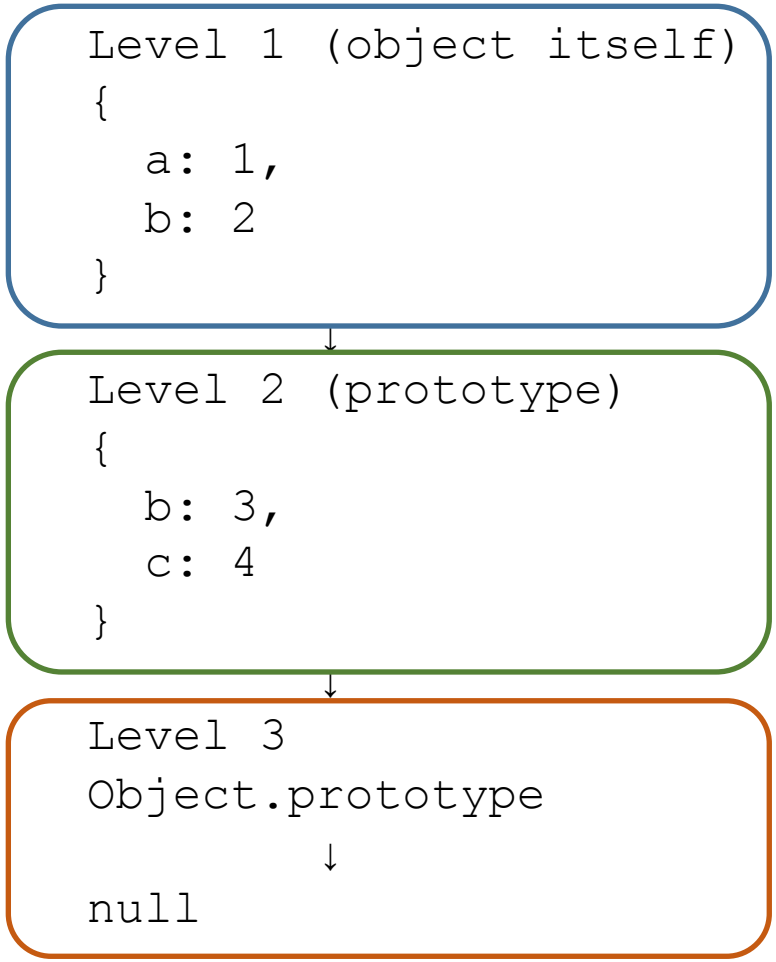
Prototype-based inheritance

```
const o = {
  a: 1,
  b: 2,
    __proto__: {
      b: 3,
      c: 4,
    },
};
```

```
console.log(o.a);

console.log(o.b);

console.log(o.c);

console.log(o.d);
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Inheritance_and_the_prototype_chain

Prototype-based inheritance



Prototype-based inheritance

```
const animal = {
  eats: true
};
```

```
const rabbit = {
  jumps: true
};
```

```
rabbit.__proto__ = animal;
```

```
console.log(rabbit.jumps);
```

```
console.log(rabbit.eats);
```

Prototype-based inheritance

- If untrusted code executing inside a sandbox obtains a reference whose prototype chain leads to objects outside the sandbox, it can traverse the chain to reach privileged functionality.

- For example, reaching the Function constructor via the root prototype allows the construction of functions that execute in the host context, enabling arbitrary code execution

Dynamic code evaluation

- JavaScript provides several mechanisms for dynamically evaluating code at runtime

- JavaScript takes a string and executes it as real code at runtime.

- The eval() function takes a string argument and executes it as JavaScript code in the current scope.

- eval() runs a string as code inside the calling scope

Dynamic code evaluation

```
let x = 10;
let secret = "myPassword123";


let result = eval("x + 1");
console.log(result);



eval("x = 99");
console.log(x);
```

Dynamic code evaluation

```
function calculate(userInput) {
let balance = 5000;

return eval(userInput);
}
```

NEVER do this, attacker controls userInput

The `this` keyword

- The value of this in JavaScript depends on the calling context:

- In a method call, this refers to the object on which the method was invoked;

- In a standalone function call in non-strict mode, this refers to the global object;

- In strict mode, it is undefined.

Calling a function on an object (Method Call)

```
const dog = {
  name: "Rex",
  bark() {
    console.log(this.name);
  }
};

dog.bark();
```

this = dog → prints "Rex"

Calling a function on its own

```
function showName() {
  console.log(this);
}

showName();
```

this = window → the entire global scope!

How attackers exploit this

```
const w = (function() {
  return this;
})();

w.eval("steal everything");
```

Calling a function on its own

```
'use strict';

function showName() {
  console.log(this);
}

showName();
```

this = undefined

Questions??

zubair.ahmad@giki.edu.pk

Office: G14 FCSE lobby